

Dictionaries, Sets & Functions

Real-World Data & Reusable Logic

Session 8

What We're Covering Today

- Dictionaries — key-value data, exactly like real APIs and databases
- Accessing, updating, and safely handling missing keys
- Nested dictionaries & lists of dictionaries — how real data actually looks
- Sets — unique values, and the four core set operations
- Why sets are faster — a real performance reason to use them
- Functions — why they exist, and how the industry actually uses them
- Scope, default arguments, and a famous Python gotcha
- Refactoring repeated code into functions — a real case study
- Industry-style tasks and two mini projects



PART 1

Dictionaries

The data structure behind almost every API, config file, and database record you'll ever touch.

What Is a Dictionary?



Labeled boxes, not numbered ones

- A dictionary stores data as key-value pairs — a label, and the value that belongs to it
- Think of a real address book: you look someone up by name, not by "the 5th contact"
- A list is organized by position (index 0, 1, 2…). A dictionary is organized by meaning (name, age, city…)
- This single shift — from position to meaning — is why dictionaries are everywhere in real software

Where You Already See This in the Real World

Contact Lists

Name → phone number, exactly like your phone

Database Rows

Column name → value, for one record

JSON from an API

Nearly every web API returns data shaped exactly like a Python dict

Config Files

Settings stored as name → value, like "volume": 80

Creating a Dictionary

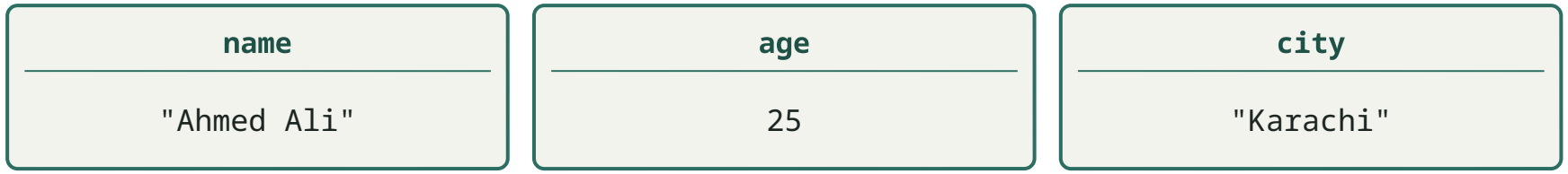
- Curly braces { } create a dictionary
- Each entry is key: value, separated by commas
- Keys are usually strings; values can be absolutely anything — even another dictionary

```
contact = {
    "name": "Ahmed Ali",
    "age": 25,
    "city": "Karachi"
}

print(type(contact))   # <class 'dict'>
```

A Dictionary, Visualized

```
contact = {"name": "Ahmed Ali", "age": 25, "city": "Karachi"}
```



- contact["name"] → "Ahmed Ali"
- Each box is labeled by its key, not by a position — there is no index 0 or 1 here at all
- This is the key visual difference from the list diagrams you've already seen



Accessing Values: [ ] vs. .get()

- contact["key"] gets the value — but crashes with a KeyError if the key doesn't exist
- .get("key") returns None instead of crashing, and you can even set your own fallback
- In real code, .get() is almost always the safer choice for data you don't fully control

```
contact = {"name": "Ahmed", "age": 25}

print(contact["name"])      # "Ahmed"
print(contact["email"])     # KeyError!

print(contact.get("email"))          # None
print(contact.get("email", "N/A"))   # "N/A"
```

Adding & Updating a Key

```
contact["email"] = "ahmed@email.com"   ->   adds a brand-new key
```

BEFORE

name	age
"Ahmed"	25



AFTER

name	age	email
"Ahmed"	25	"ahmed@..."

- The same syntax also updates an existing key — contact["age"] = 26 simply overwrites the old value
- **Unlike a tuple, a dictionary is fully mutable — this is exactly why it's so useful for live, changing data**

Removing a Key

- del removes a key completely — no value is returned
- pop(key) removes a key AND returns its value, just like list.pop()
- popitem() removes and returns the last inserted key-value pair

```
contact = {"name": "Ahmed", "age": 25, "city": "Karachi"}

del contact["city"]
print(contact)   # {'name': 'Ahmed', 'age': 25}

age = contact.pop("age")
print(age)       # 25
```

Checking Existence: in / not in

- in checks whether a key exists — not a value — this trips people up at first
- Always check before accessing a key you're not sure exists, unless you're using .get()

```
contact = {"name": "Ahmed", "age": 25}

print("name" in contact)     # True
print("email" in contact)    # False
print(25 in contact)         # False! (25 is a value, not a key)
```

.keys(), .values(), .items()

- .keys() gives you every key, .values() gives you every value
- .items() gives you both together, as key-value pairs — the most commonly used of the three

```
contact = {"name": "Ahmed", "age": 25, "city": "Karachi"}

print(contact.keys())    # dict_keys(['name', 'age', 'city'])
print(contact.values())  # dict_values(['Ahmed', 25, 'Karachi'])
print(contact.items())   # dict_items([('name', 'Ahmed'), ...])
```



Looping Over a Dictionary

- Looping directly over a dict gives you just the keys
- Looping over .items() gives you both the key and value together — this is what you'll use most

```
contact = {"name": "Ahmed", "age": 25, "city": "Karachi"}

for key, value in contact.items():
    print(key, "->", value)

# name -> Ahmed
# age -> 25
# city -> Karachi

for key in contact.keys():
    print(key)

for val in contact.values():
    print(val)
```

Real-World Use: A Word Frequency Counter

- A dictionary is the natural tool for counting anything — a classic, genuinely industry-relevant use case
- This exact pattern is the foundation of real text analytics and search engines

```
text = "the cat sat on the mat the cat ran"
words = text.split()

counts = {}
for word in words:
    if word in counts:
        counts[word] += 1
    else:
        counts[word] = 1

print(counts)   # {'the': 3, 'cat': 2, 'sat': 1, ...}
```

Nested Dictionaries — Real Records

- A dictionary can hold another dictionary as a value — this is how real, structured data is represented
- This is exactly the shape of data returned by most web APIs

```
user = {
   "name": "Ahmed Ali",
   "address": { "city": "Karachi", "zip": "74200“ }
}
print(user["address"]["city"])   # "Karachi“
```

Lists of Dictionaries — How Real Data Actually Looks

- This exact structure is what a CSV file, a database table, or an API result usually becomes once loaded into Python
- This is precisely what pandas will build on top of later in the course

```
students = [
    {"name": "Ahmed", "score": 88},
    {"name": "Sara", "score": 92},
    {"name": "Bilal", "score": 79},
]

for student in students:
    print(student["name"], "->", student["score"])
```

Merging Dictionaries: .update() and |

- .update() adds all key-value pairs from one dictionary into another, overwriting shared keys
- The | operator (Python 3.9+) does the same thing, but returns a new dictionary instead of changing one in place

```
defaults = {"theme": "light", "font_size": 12}
user_settings = {"font_size": 16}

defaults.update(user_settings)
print(defaults)   # {'theme': 'light', 'font_size': 16}

merged = defaults | {"theme": "dark"}
```



.setdefault() — Avoiding KeyError Gracefully

- setdefault(key, default) returns the value if the key exists — and creates it with that default if it doesn't
- A clean one-line alternative to writing an if/else just to handle a possibly-missing key

```
counts = {}

counts.setdefault("apple", 0)
counts["apple"] += 1
print(counts)   # {'apple': 1}

# calling it again on an existing key leaves it untouched
counts.setdefault("apple", 0)
print(counts)   # still {'apple': 1}
```

Practice Set — Dictionaries

Try these yourself first — no code shown, that's the point

1. Create a dictionary representing yourself: name, age, and favorite subject
2. Use .get() to safely look up a key that might not exist, with a fallback value
3. Add a new key to your dictionary, then update an existing one
4. Loop through your dictionary using .items() and print each key and value
5. Build a small word-count dictionary from a sentence of your choice



PART 2

Sets

Every value appears exactly once — nothing more, nothing less.

What Is a Set?



No order.
No duplicates.
Ever.

- A set stores a collection of values where every value is guaranteed to be unique
- Think of a guest list where writing someone's name twice doesn't invite them twice
- Sets have no order and no index — you can't ask for "the first item"
- Real uses: unique tags on a post, unique visitor IDs, unique words in a document



Creating a Set

- Curly braces create a set — but watch out: {} alone creates an empty dictionary, not a set
- set() is the safe way to create an empty set

```
tags = {"python", "data", "beginner"}
print(type(tags))    # <class 'set'>

empty_dict = {}
print(type(empty_dict))   # <class 'dict'>  <- gotcha!

empty_set = set()
print(type(empty_set))    # <class 'set'>
```



Why Duplicates Disappear

- Passing a list into set() automatically removes every duplicate, keeping only unique values
- This single behavior is the whole reason sets exist

```
numbers = [1, 2, 2, 3, 3, 3, 4]
unique = set(numbers)
print(unique)   # {1, 2, 3, 4}
```

Adding & Removing From a Set

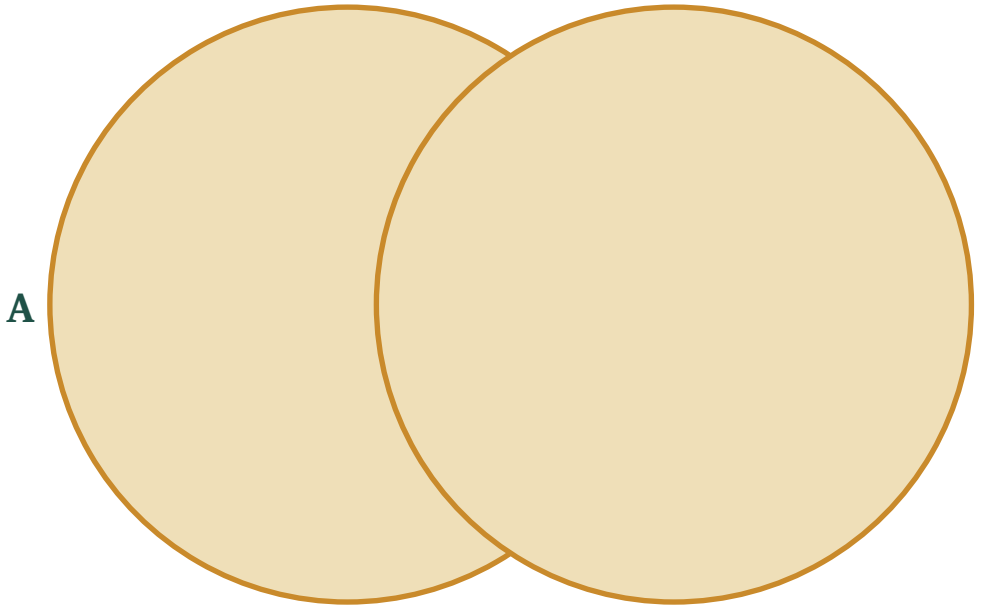
- add() inserts a single value
- remove() deletes a value, but raises an error if it isn't there
- discard() does the same, but stays silent if the value is missing — often the safer choice

```
tags = {"python", "data"}

tags.add("beginner")
tags.remove("data")

tags.discard("not_here")   # no error, even though it's missing
tags.remove("not_here")    # KeyError!
```

Set Operation: Union (|)

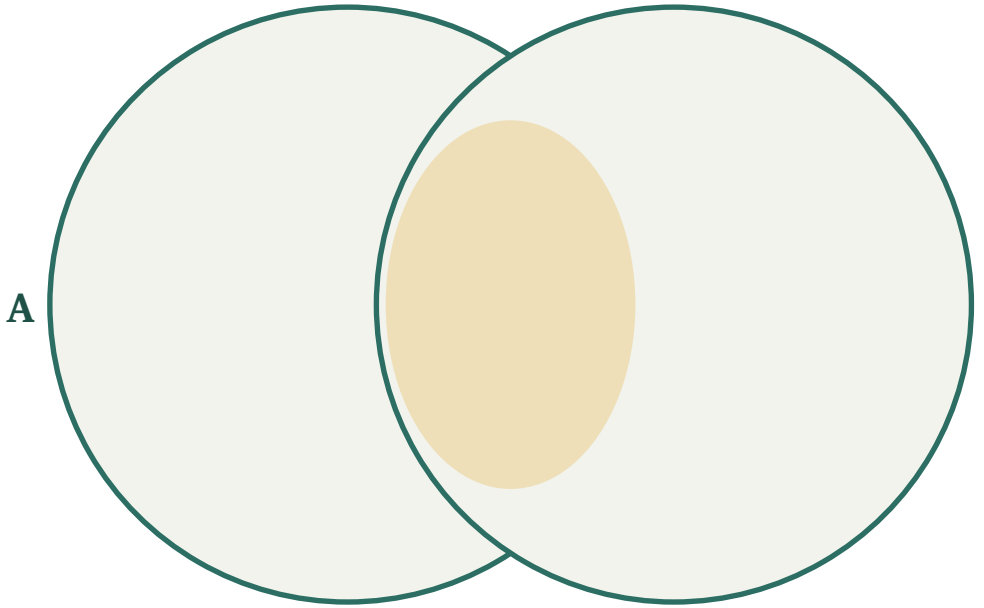



```
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b)
# {1, 2, 3, 4, 5}
```

- Union combines everything from both sets, with duplicates automatically removed.

Set Operation: Intersection (&)

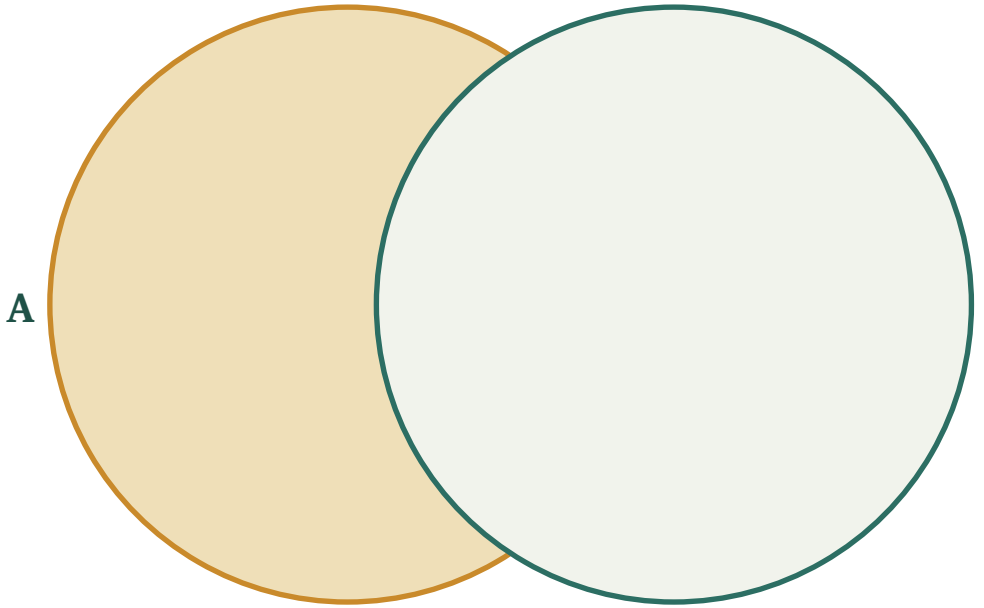



```
a = {1, 2, 3}
b = {3, 4, 5}

print(a & b)
# {3}
```

- Intersection keeps only the values that exist in BOTH sets — the shared middle.

Set Operation: Difference (-)



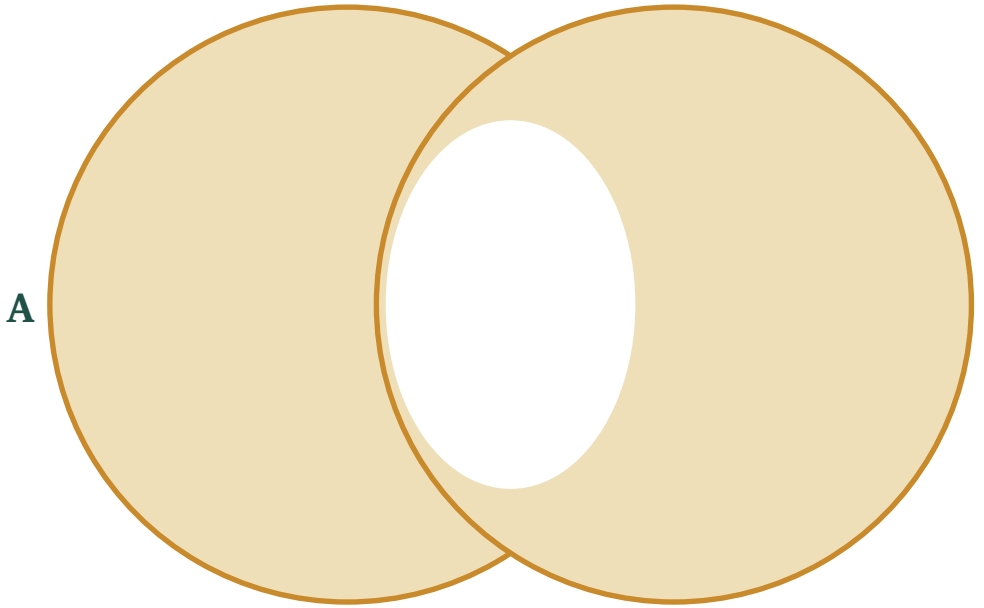

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a - b)
# {1, 2}
```

- Difference keeps only what's in A but NOT in B — order matters here, a - b differs from b - a.

Set Operation: Symmetric Difference (^):

- Give me everything that is NOT shared between A and B




```
a = {1, 2, 3}
b = {3, 4, 5}

print(a ^ b)
# {1, 2, 4, 5}
```

- Symmetric difference keeps everything EXCEPT what's shared — the opposite of intersection.

Real-World Use: Removing Duplicates From a List

- This is one of the single most common real uses of a set in everyday code
- Converting a list to a set and back instantly removes every duplicate
- Note: this does not preserve the original order — use with care if order matters

```
emails = ["a@x.com", "b@x.com", "a@x.com", "c@x.com"]

unique_emails = list(set(emails))
print(unique_emails)
# ['a@x.com', 'c@x.com', 'b@x.com']  (order not guaranteed)
```

Real-World Use: Fast Membership Testing



in on a List

Checks every item one by one — slower as the list grows



in on a Set

Near-instant, regardless of size — this is why sets exist for lookups



Where Are Sets Actually Used?

Data Cleaning — removing duplicate records, IDs, categories, or values

Fast Searching — quickly checking whether an item exists

Maps & Navigation — tracking visited locations during route-finding and graph search algorithms such as **A*** and **Dijkstra's**

AI Search — tracking visited states/nodes in **BFS, DFS, A*** and other graph-based algorithms

NLP — finding unique words, tokens, keywords, or vocabulary

Recommendation Systems — comparing users' interests, skills, products, or preferences

Security & Access Control — managing unique permissions, blocked users, or allowed resources

Data Science — comparing datasets, finding common/missing categories, and identifying unique values

Practice Set — Sets

Try these yourself first — no code shown, that's the point

1. Create a set of 5 numbers that includes at least one duplicate, and print it

2. Remove duplicates from a list of your own using set()

3. Create two sets of your own and find their union and intersection

4. Find the difference between two sets of your own, in both directions



PART 3

Functions

The single biggest jump in this course so far — and the foundation of every real codebase.

Why Functions Exist — The Industry Case



Write it once. Fix it once.

- Function is a block of code that can be written once and used anytime anywhere when u need it.
- Imagine writing the same 10-line email-checking logic in 5 different places in a real project
- Now imagine finding a bug in it — you'd need to fix it in all 5 places, and it's easy to miss one
- A function means that logic exists in exactly one place — write once, reuse everywhere, fix once
- Every real, professional codebase is fundamentally just functions calling other functions

The Recipe Analogy



Ingredients in, dish out

- A function is like a recipe: you give it ingredients (parameters), it follows the same steps every time, and it produces a result
- You don't need to know how the kitchen works to use the recipe — you just need to know what to put in, and what comes out
- This hiding of internal detail is called abstraction — a core idea in real software design



def — Basic Syntax

- def starts a function definition, followed by a name and parentheses
- The indented block underneath only runs when the function is actually called
- Defining a function does not run it — calling it does

```
def greet():
    print("Hello!")

# nothing happens yet — it's only defined

greet()   # NOW it runs
# Hello!
```



Parameters vs. Arguments



Parameter

- The name used INSIDE the function definition
- It's a placeholder — a variable waiting to receive a value
- Example: the name in def greet(name):



Argument

- The actual value you pass in WHEN CALLING the function
- This is what fills the placeholder
- Example: the "Ahmed" in greet("Ahmed")

return vs. print — The Classic Confusion

- print() just displays something on screen — the value is gone once shown
- return actually hands a value back to whoever called the function, so it can be stored and reused
- A function with only print() cannot have its result saved into a variable

```
def add_v1(a, b):
    print(a + b)

def add_v2(a, b):
    return a + b

result1 = add_v1(2, 3)   # prints 5, but result1 is None
result2 = add_v2(2, 3)   # nothing printed, but result2 is 5
```

Functions With No Return

- If a function never hits a return statement, it automatically returns None
- This is exactly what happened to result1 on the previous slide

```
def greet(name):
    print("Hello,", name)

result = greet("Ahmed")
print(result)   # None
```



Multiple Parameters

- A function can accept as many parameters as it needs, separated by commas
- Arguments are matched to parameters in order, by default

```
def describe_student(name, age, city):
    return f"{name}, age {age}, from {city}"

print(describe_student("Ahmed", 25, "Karachi"))
# "Ahmed, age 25, from Karachi"
```



Default Parameter Values

- A parameter can have a default value, used automatically if the caller doesn't provide one
- This is exactly how real-world functions offer sensible behavior out of the box, while still being customizable

```
def calculate_price(price, discount=0.1):
    return price - (price * discount)

print(calculate_price(100))        # 90.0
(uses default 10%)
print(calculate_price(100, 0.25))  # 75.0
(overridden)
```



Keyword Arguments

- You can call a function using name=value, instead of relying on argument order
- This makes function calls far more readable, especially with many parameters

```
def describe_student(name, age, city):
    return f"{name}, age {age}, from {city}"

print(describe_student(city="Karachi",
name="Ahmed", age=25))
# order doesn't matter when you use keyword
arguments
```



Returning Multiple Values

- A function can return more than one value at once, separated by commas
- Python quietly packs them into a tuple — the exact tuple unpacking you already know handles the rest

```
def get_min_max(numbers):
    return min(numbers), max(numbers)

lowest, highest = get_min_max([4, 8, 15, 16, 23])
print(lowest, highest)   # 4 23
```

Scope: Local vs. Global Variables

- A variable created inside a function is local — it only exists while that function is running, then disappears
- A variable created outside any function is global — visible everywhere
- A function can READ a global variable, but assigning to a variable inside a function makes it local by default, even if a global with the same name exists

```
x = 10  # global

def show_x():
    print(x)  # can read the global x just fine

def change_x():
    x = 5     # this creates a NEW local x — the global x is untouched

change_x()
print(x)  # still 10
```

The Mutable Default Argument Trap: Real world known interview Question

- A famous, real Python gotcha: using a mutable object (like a list) as a default value
- The default is created only ONCE, when the function is defined — not every time it's called
- Every call that relies on the default ends up sharing the exact same list

Rule to remember: Never use a mutable object (list, dict, set, etc.) as a default function argument.

```
# BROKEN
def add_item(item, cart=[]):
    cart.append(item)
    return cart

print(add_item("apple"))    # ['apple']
print(add_item("banana"))   # ['apple', 'banana']  <- unexpected!

# FIXED
def add_item_safe(item, cart=None):
    if cart is None:
        cart = []
    cart.append(item)
    return cart
```

Docstrings — Writing Professional Functions

- A docstring is a short description written right under def, in triple quotes
- Real, professional code is expected to have these — they show up automatically in help() and documentation tools
- A good docstring explains what the function does, its parameters, and what it returns

```
def calculate_price(price, discount=0.1):
    """
    Calculate a discounted price.

    price: the original price
    discount: fraction to subtract (default 10%)
    Returns: the final price after discount
    """
    return price - (price * discount)

print(calculate_price.__doc__)

help(calculate_price)
```

Functions Calling Other Functions

- A function can call another function inside it — this is called composition
- Real programs are built almost entirely this way: small, focused functions combined into bigger behavior

```
def is_valid_email(email):
    return "@" in email and email.endswith(".com")

def register_user(name, email):
    if not is_valid_email(email):
        return "Invalid email"
    return f"Registered {name}"

print(register_user("Ahmed", "ahmed@mail.com"))
```



Case Study: Refactoring Repeated Code

BEFORE — the same logic, copy-pasted three times

```
price1 = 100
final1 = price1 - (price1 * 0.1)
print(final1)

price2 = 250
final2 = price2 - (price2 * 0.1)
print(final2)

price3 = 80
final3 = price3 - (price3 * 0.1)
print(final3)
```

AFTER — one function, reused

```
def apply_discount(price, rate=0.1):
    return price - (price * rate)

print(apply_discount(100))
print(apply_discount(250))
print(apply_discount(80))

# one place to fix a bug,
# not three
```

Capstone: Sorting a Dictionary by Value

- sorted() can sort a dictionary's items — but it needs to be told WHAT to sort by
- A small helper function tells sorted() exactly that, using its key parameter
- This single pattern — a helper function plus sorted() — is genuinely used everywhere in real data work
- Key means "Use this function to decide what determines the sorting order.

```
scores = {"Ahmed": 88, "Sara": 92, "Bilal": 79}

def get_score(pair):
    return pair[1]

ranked = sorted(scores.items(), key=get_score, reverse=True)
print(ranked)
# [('Sara', 92), ('Ahmed', 88), ('Bilal', 79)]
```



Practice Set — Functions

Try these yourself first — no code shown, that's the point

1. Write a function that takes a name and returns a greeting sentence
2. Write a function with a default parameter, and call it both with and without providing that argument
3. Write a function that returns two values at once, and unpack them into two variables
4. Deliberately trigger the mutable default argument bug, then fix it
5. Write a short docstring for one of your own functions



INDUSTRY-STYLE TASKS

Build the Things You'll Actually Be Asked to Build

Not toy examples — inventories, deduplication, and reusable validators are real, everyday work.

Today's Tasks — Part A: Dictionaries

Small tasks — do them in order

1. Build a simple inventory system: a dictionary of product names to stock counts, then add, update, and remove a few items

2. You're given a dictionary shaped like a fake API response (nested keys) — extract and print 3 specific nested values

3. Build a word-frequency counter on a full paragraph of your choice, then print the 3 most common words using sorted()

Today's Tasks — Part B: Sets

Small tasks — do them in order

4 You have two lists of email addresses with overlaps — use sets to build one clean, duplicate-free list

5 You have two lists of user IDs from two different systems — use sets to find which users exist in both

6 You have one list of user IDs — use sets to find which ones are NOT in a second "banned" list

Today's Tasks — Part C: Functions

Small tasks — do them in order

7 Write a reusable is_valid_password(password) function: valid only if 8+ characters and contains a digit

8 Write a calculate_discount(price, rate=0.1) function, and use it to replace 3 repeated manual calculations

9 Take a piece of your own repeated code from an earlier session, and refactor it into one function

10 core tasks in total — due before our next class



PART D

Put It All Together

Two projects combining dictionaries, sets, and functions — genuinely close to real early-career tasks.

Mini Project 1: Contact Book Manager



Dict + functions, combined

- Store contacts as a dictionary of dictionaries: {"Ahmed": {"phone": "...", "city": "..."}, ...}
- Write separate functions: add_contact(), find_contact(), remove_contact() — each one operating on the same contacts dictionary
- Use .get() inside find_contact() so looking up a missing name never crashes the program
- This is genuinely how small real applications are structured — data plus functions that operate on it

Mini Project 2: Student Grade Analyzer



Lists of dicts, sets & functions

- You're given a list of dictionaries: [{"name": "Ahmed", "grade": "A"}, {"name": "Sara", "grade": "B"}, ...]
- Write a function that returns how many students got each grade, as a dictionary
- Use a set to find and print the unique grade levels that appear at all
- Write a function that returns the names of every student with an "A", using the accumulator pattern from Loops



RECAP

Dictionaries and sets are how real data is actually shaped.
Functions are how real code actually stays maintainable.

*Next: advanced functions — *args, **kwargs, lambda, recursion, and closures.*